

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Introducción a la Computación 22.06

Calculadora

Trabajo de Laboratorio N ° 06

Curso 2020A – 1er Cuatrimestre

GRUPO N°: 04		INTEGRANTES
Legajo N°	Nombre	
62490	Marcos Dario Alegre	
62012	Santiago Feldman	
62447	Anna Candela Gioia Perez	
61758	Juan Sebastián Helou	

	Fecha	Docente
Realizado	28/10/2020	
Presentado	03/11/2020	
Aprobado		

Diagramas de flujo

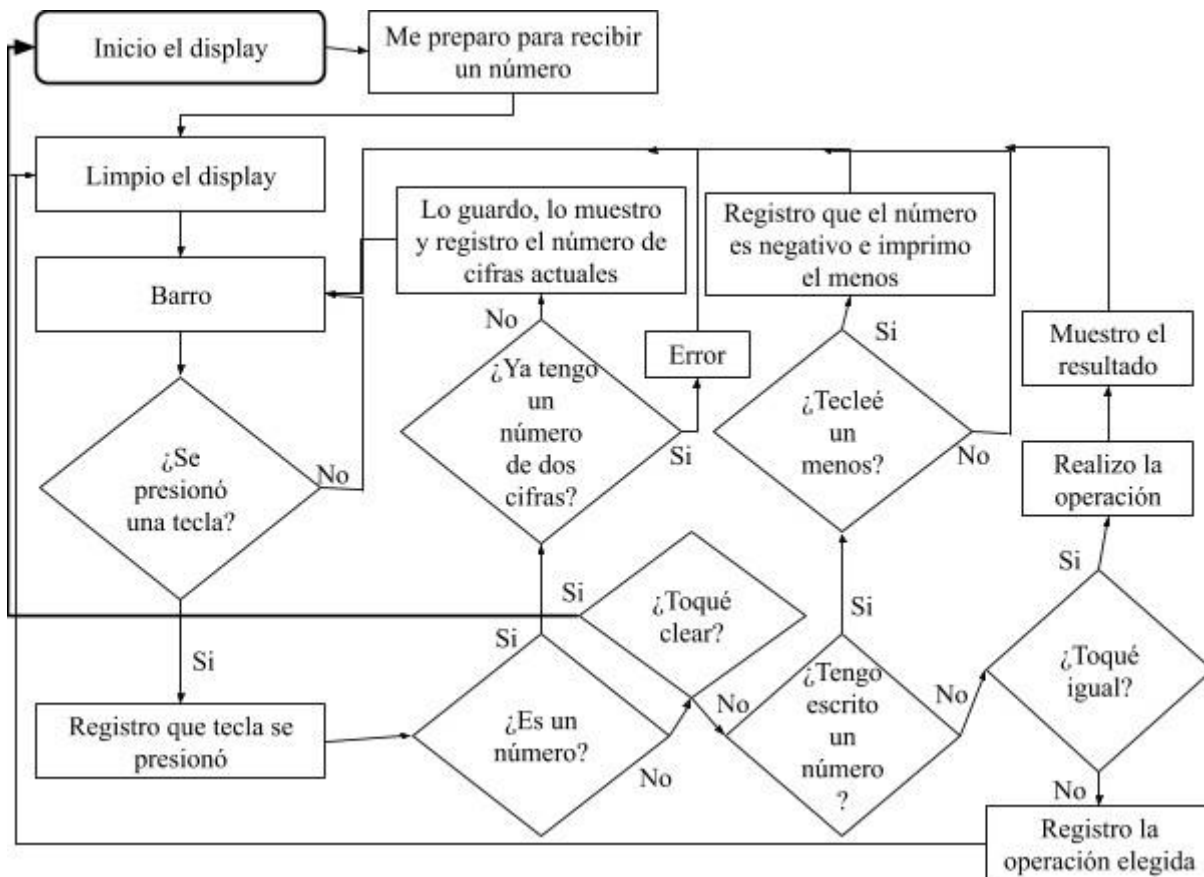


Diagrama de flujo del MAIN PROGRAM

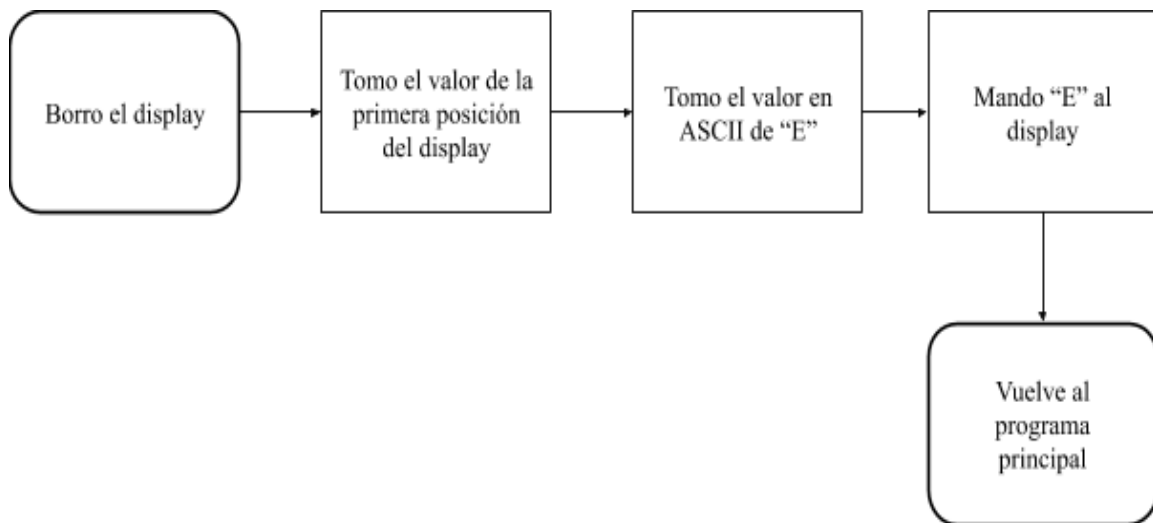


Diagrama de flujo de ERROR

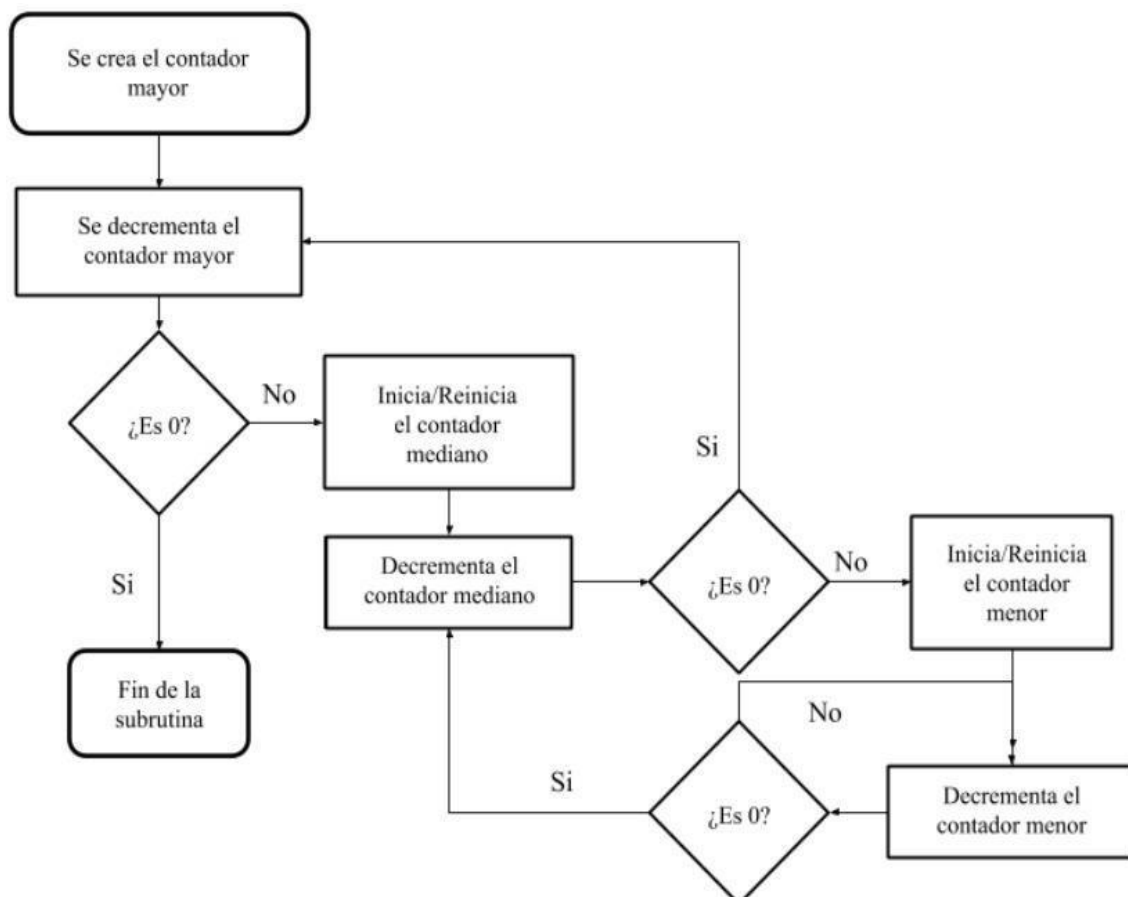


Diagrama de DELAY

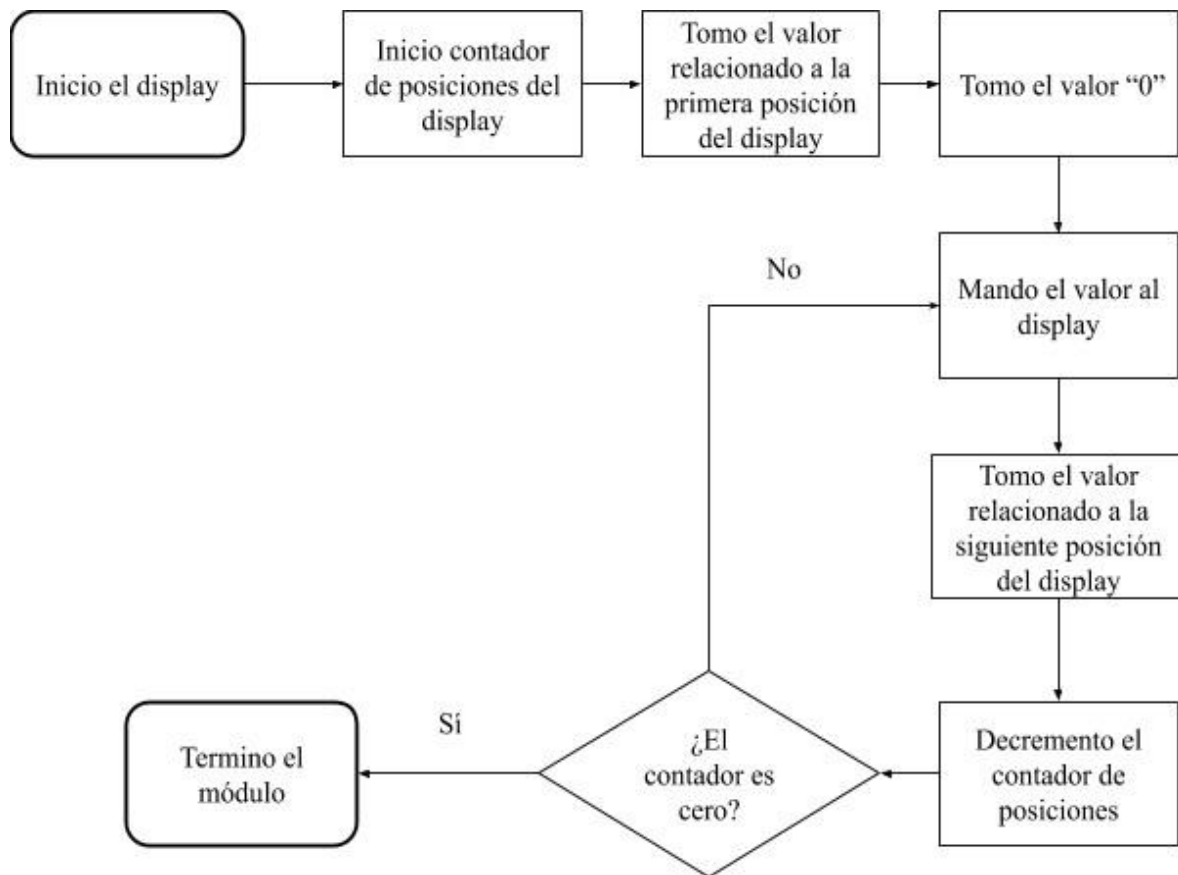


Diagrama de flujo que describe el borrado del display

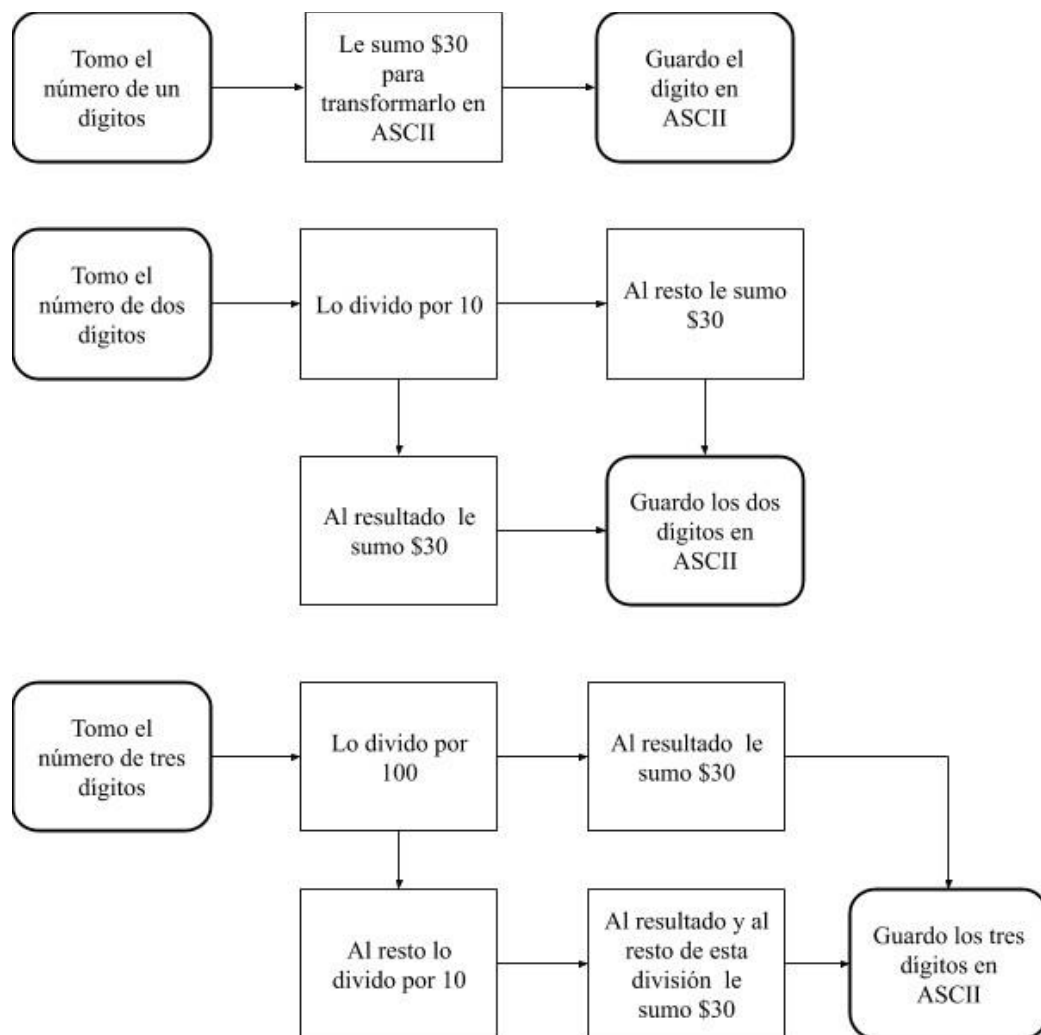


Diagrama de flujo que describe la transformación a ASCII de los resultados

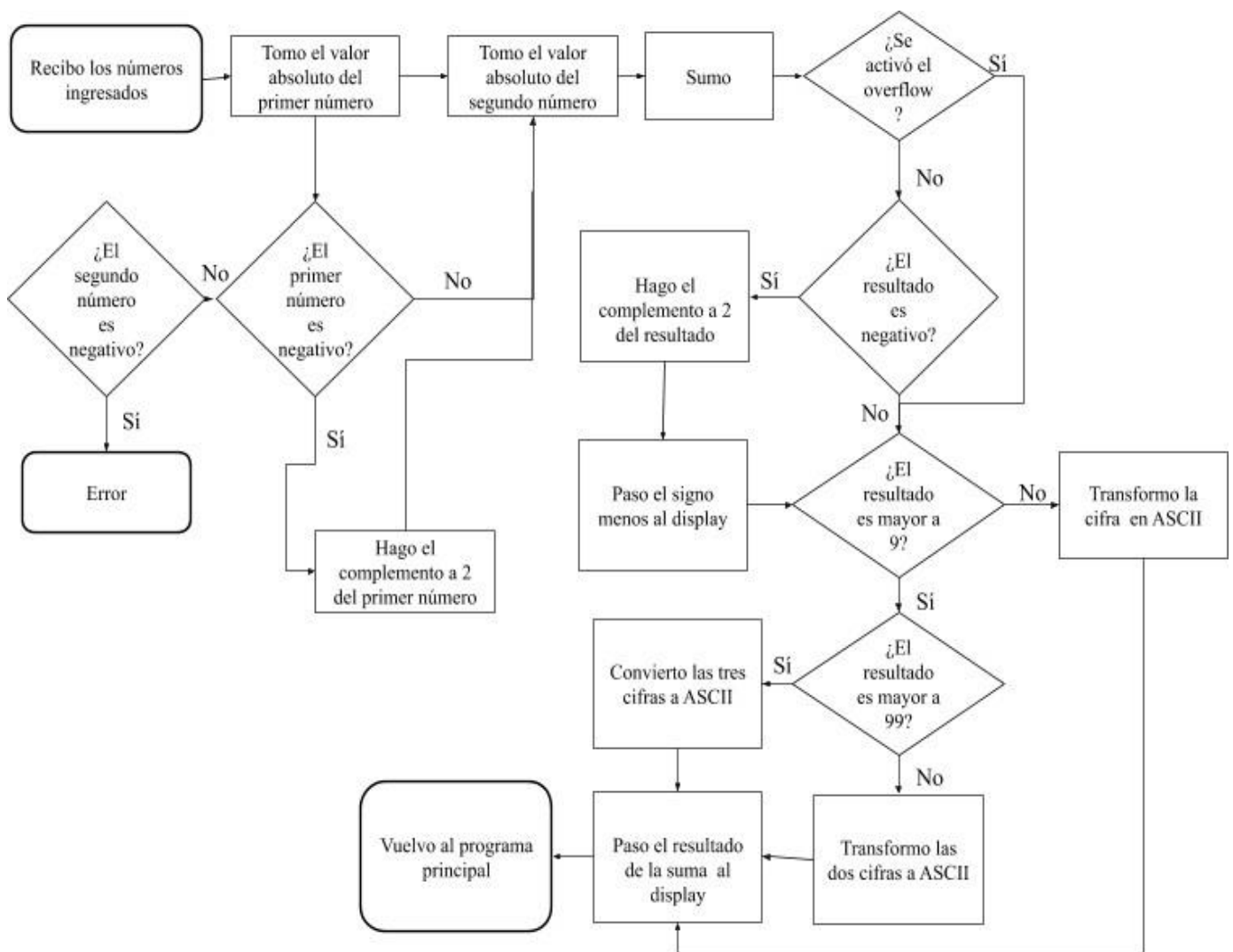


Diagrama de flujo de SUMAC

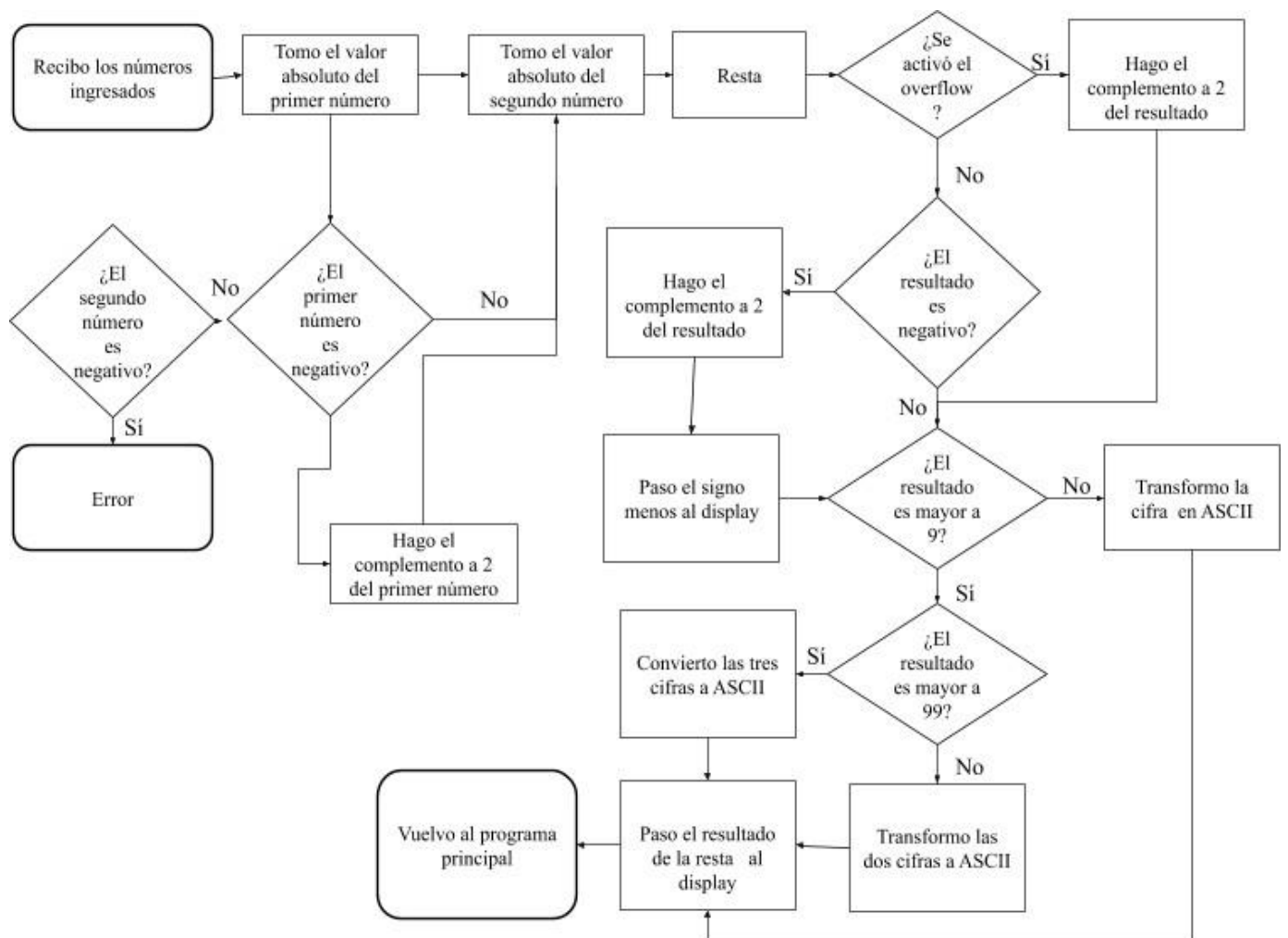


Diagrama de flujo de RESTAC

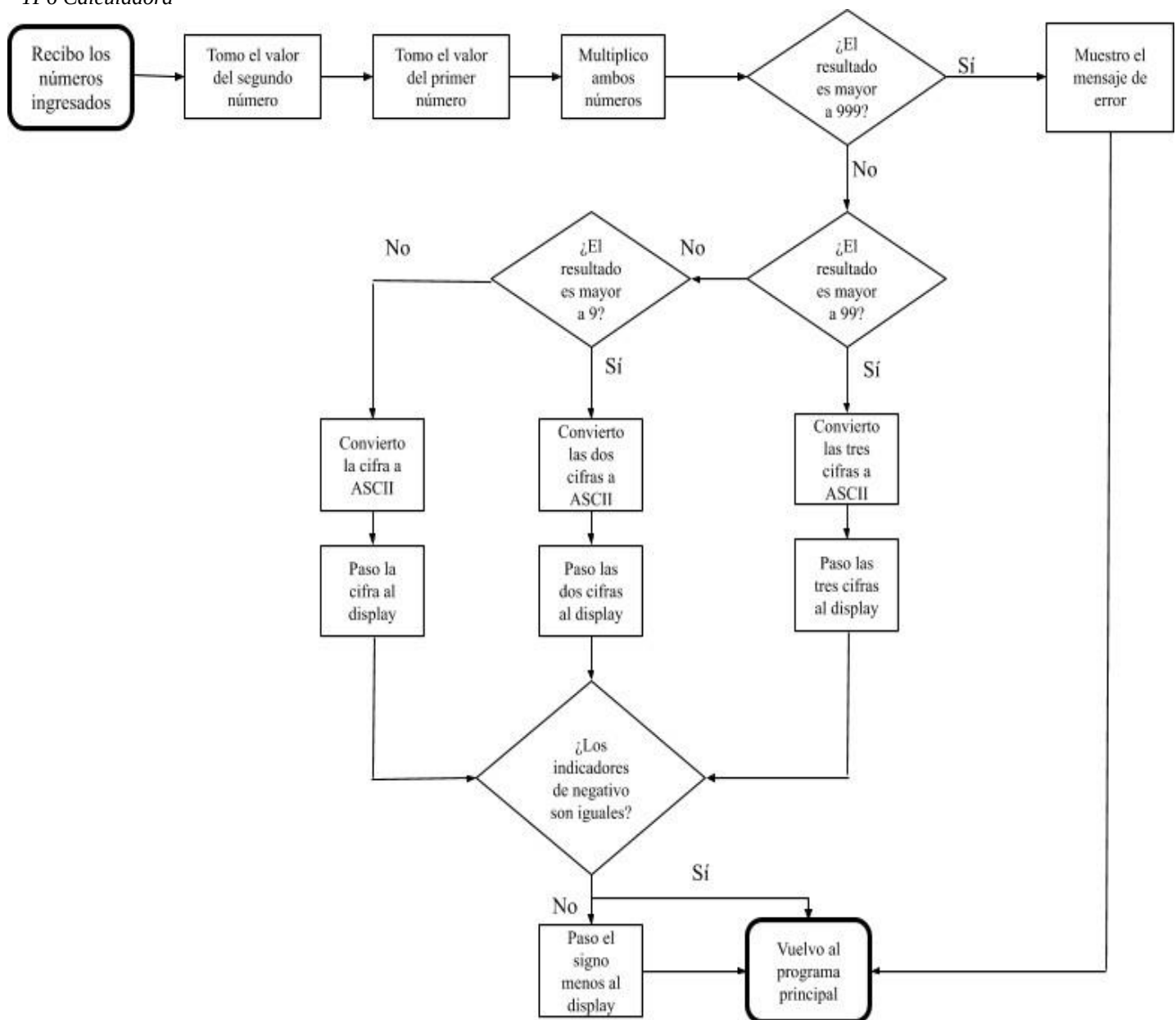


Diagrama de flujo de PROD

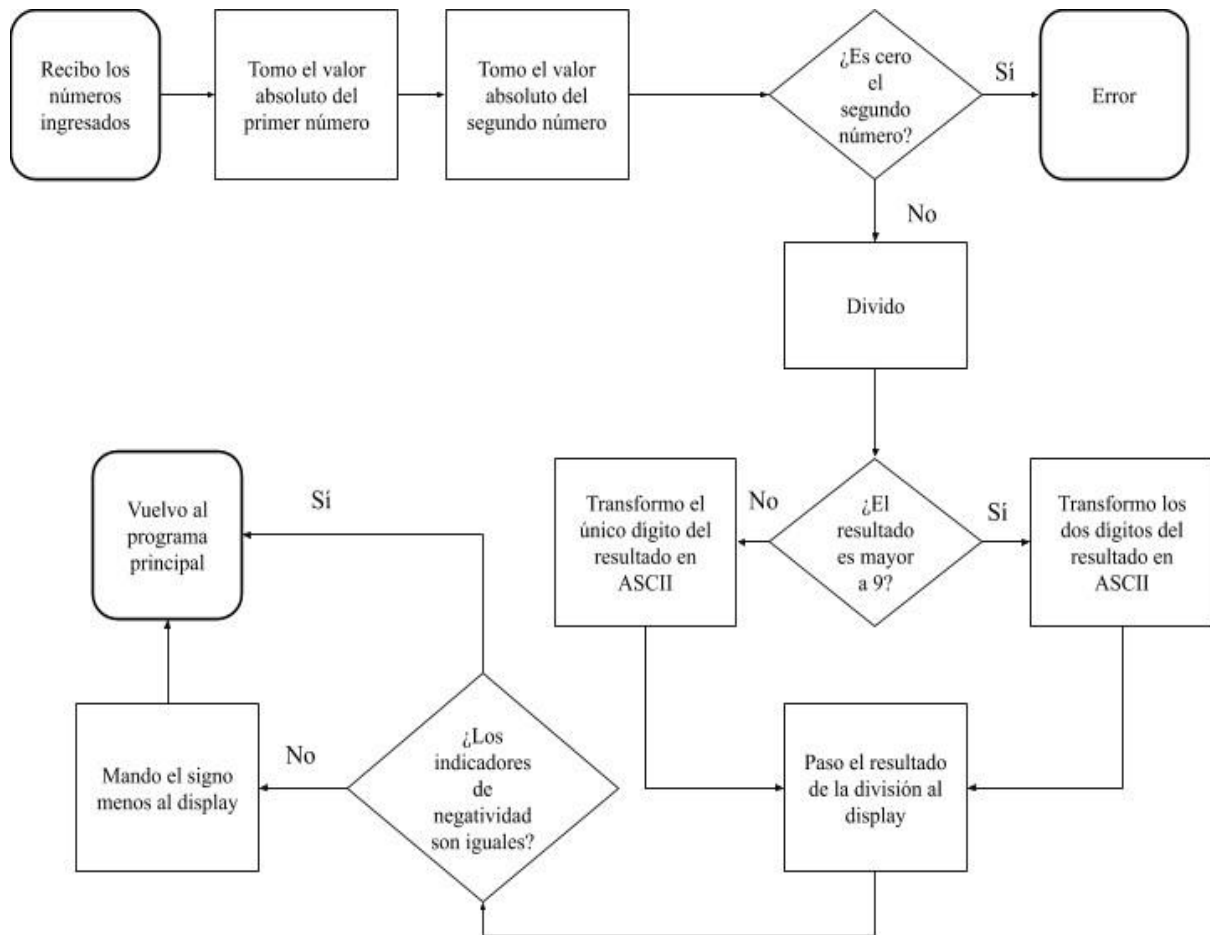


Diagrama de flujo de DIVI

TP6J (MAIN PROGRAM)

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           28/Oct/20  00:08:51  #
#
#   Source   =   tp6j.msa                                #
#   List     =   tp6j.lst                                #
#   Object   =   tp6j.r07                                #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                                p68h11
2
3  0000                                extern  show,division,multi,sumar,restar
4  0000                                public  counter,msg,stackP,flagn1,flagn2,flags,flagr,outchar
5
6                                $display2.def
7                                *****Ports*****
8
9  1007                                DDRC            equ    $1007            ;data direction register for C
10 1002                                PIOC            equ    $1002            ;for strobe B assertion change
11 1005                                PORTCL          equ    $1005            ;port C latched
12 1004                                PORTB           equ    $1004            ;port B
13 1003                                PORTC           EQU    $1003            ;rows
14
15                                *****Cols and Row Ports*****
16
17 1003                                ROWS            EQU    PORTC            ;outputs
18 1005                                COLS            EQU    PORTCL          ;inputs
19
20                                *****Row Masks*****
21
22 000E                                FIL0            EQU    00001110b        ;mascaras para manejo de filas
23 000D                                FIL1            EQU    00001101b
24 000B                                FIL2            EQU    00001011b
25 0007                                FIL3            EQU    00000111b
26
27                                *****Keyboard Mask*****
28
29
30 0003                                DI_START        EQU    $03
31 0000                                DI_END          EQU    $00
32
33 0003                                L                EQU    $3
34 00FF                                M                EQU    $FF
35 00FF                                S                EQU    $FF
36
37
38 0000                                rseg            CODE
39
40
41 0000 BD001A                        jsr            initdis            ;se inicializa el display
```

```

42
43 0003 8E0063    clearall    lds      #stackP
44
45 0006 7F0005          clr      flagn1
46 0009 7F0006          clr      flagn2
47 000C 7F0001          clr      flags      ;se reinician los flags de operacion
48 000F 7F0002          clr      flagr
49 0012 7F0003          clr      flagm
50 0015 7F0004          clr      flagd
51
52 0018 C603      clear      ldab     #DI_START      ;toma el valor de la primera posición del
                                           ;display
53 001A CE0004          ldx      #4              ;inicia contador de posiciones del display
54 001D 8600          ldaa     #0              ;toma el valor 0 para borrar el display
55 001F BD002A      cleandisp  jsr      outchar      ;llama a la subrutina para poner los valores
                                           ;del display
56 0022 5A          decb     ;decrementa la posición del display
57 0023 09          dex      ;decrementa el contador de posiciones
58 0024 26F9          bne     cleandisp      ;si el contador no es 0 repite la secuencia
59
60 0026 8602      primedisp  ldaa     #2
61 0028 B70000          staa    counter      ;se preparan las posiciones de memoria que
                                           ;funcionan
62 002B 8600          ldaa     #0              ;como contador y como retenedor de dígitos
63 002D B70008          staa    msg
64
65
66
67 *****
68 *          SECCION DE LECTURA          *
69 *          *                             *
70 * Esta parte del programa se encarga *
71 * de leer constantemente los valores *
72 * del Puerto C para fijarse si alguna *
73 * tecla es presionada                *
74 *****
75
76 0030 BD0000      barrido    jsr      delay      ;se genera un poco de retraso en el barrido
77                                           ;para evitar presionar involuntariamente la
                                           ;misma tecla más de una vez
78
79 0033 C60E          ldab     #FIL0
80 0035 F71003          stab    PORTC      ;se prepara la máscara de la fila 0
81
82 *****;SE PRESIONA EL BOTON
83
84 0038 B61003          ldaa     PORTC
85 003B 81EE          cmpa     #11101110b
86 003D 270F          beq      read0
87 003F 81DE          cmpa     #11011110b
88 0041 270E          beq      read1      ;se compara el valor del puerto c con los
89 0043 81BE          cmpa     #10111110b      ;valores correspondientes a cada tecla de la
90 0045 270D          beq      read2      ;de la fila 0 detectar si se presionó una
                                           ;tecla
91 0047 817E          cmpa     #01111110b
92 0049 270C          beq      read3
93 004B 7E005A          jmp     barrido1
94
95 004E 7E00CF      read0      jmp     write0
96 0051 7E00D8      read1      jmp     write1      ;se pasa a escribir el digito
97 0054 7E00E1      read2      jmp     write2      ;correspondiente a la tecla presionada
98 0057 7E00EA      read3      jmp     write3
99
100
101

```

```

102 005A C60D      barrido1      ldab    #FIL1
103 005C F71003          stab    PORTC      ;se prepara la máscara de la fila 1
104
105      *****;SE PRESIONA EL BOTON
106
107 005F B61003          ldaa    PORTC
108 0062 81ED          cmpa    #11101101b
109 0064 270F          beq     read4
110 0066 81DD          cmpa    #11011101b
111 0068 270E          beq     read5      ;se compara el valor del puerto c con los
112 006A 81BD          cmpa    #10111101b      ;valores correspondientes a cada tecla de la
113 006C 270D          beq     read6      ;fila 1 para detectar si se presionó una
                                           ;tecla
114 006E 817D          cmpa    #01111101b
115 0070 270C          beq     read7
116 0072 7E0081        jmp     barrido2
117
118 0075 7E00F3      read4      jmp     write4
119 0078 7E00FC      read5      jmp     write5      ;se pasa a escribir el dígito
120 007B 7E0105      read6      jmp     write6      ;correspondiente a la tecla presionada
121 007E 7E010E      read7      jmp     write7
122
123
124 0081 C60B      barrido2      ldab    #FIL2
125 0083 F71003          stab    PORTC      ;se prepara la máscara de la fila 2
126
127      *****;SE PRESIONA EL BOTON
128
129 0086 B61003          ldaa    PORTC
130 0089 81EB          cmpa    #11101011b
131 008B 270F          beq     read8
132 008D 81DB          cmpa    #11011011b
133 008F 270E          beq     read9      ;se compara el valor del puerto c con los
134 0091 81BB          cmpa    #10111011b      ;valores correspondientes a cada tecla de la
135 0093 270D          beq     readsuma      ;fila 2 para detectar si se presionó una
                                           ;tecla
136 0095 817B          cmpa    #01111011b
137 0097 270C          beq     readresta
138 0099 7E00A8        jmp     barrido3
139
140 009C 7E0117      read8      jmp     write8      ;se pasa a escribir el dígito
141 009F 7E0120      read9      jmp     write9      ;correspondiente a la tecla presionada
142 00A2 7E0129      readsuma    jmp     wsuma      ;se pasa a registrar el operando
                                           ;seleccionado
143 00A5 7E013B      readresta    jmp     wresta
144
145
146
147 00A8 C607      barrido3      ldab    #FIL3
148 00AA F71003          stab    PORTC      ;se prepara la máscara de la fila 3
149
150      *****;SE PRESIONA EL BOTON
151
152 00AD B61003          ldaa    PORTC
153 00B0 81E7          cmpa    #11100111b
154 00B2 270F          beq     readclear
155 00B4 81D7          cmpa    #11010111b
156 00B6 270E          beq     readdivision      ;se compara el valor del puerto c con los
157 00B8 81B7          cmpa    #10110111b      ;valores correspondientes a cada tecla de la
158 00BA 270D          beq     readmulti      ;fila 3 para detectar si se presionó una
                                           ;tecla
159 00BC 8177          cmpa    #01110111b
160 00BE 270C          beq     readigual
161 00C0 7E0030        jmp     barrido
162

```

```

163 00C3 7E0003    readclear    jmp    clearall
164 00C6 7E0188    readdivision jmp    wdivision    ;se pasa a registrar el operando
                                   ;seleccionado
165 00C9 7E0176    readmulti    jmp    wmulti
166 00CC 7E019A    readigual    jmp    wigual
167
168
169
170 *****
171 *          SECCION DE ESCRITURA          *
172 *                                          *
173 * Esta parte del programa se encarga *
174 * de escribir los valores leidos por *
175 * el teclado e impriirlos en el      *
176 * display                               *
177 *****
178
179 00CF                                rseg    CODE
180
181 00CF 8600    write0    ldaa    #0
182 00D1 36                                psha
183 00D2 BD0000                                jsr    show
184 00D5 7E0030                                jmp    barrido
185
186 00D8 8601    write1    ldaa    #1
187 00DA 36                                psha
188 00DB BD0000                                jsr    show
189 00DE 7E0030                                jmp    barrido
190
191 00E1 8602    write2    ldaa    #2
192 00E3 36                                psha
193 00E4 BD0000                                jsr    show
194 00E7 7E0030                                jmp    barrido
195
196 00EA 8603    write3    ldaa    #3
197 00EC 36                                psha
198 00ED BD0000                                jsr    show
199 00F0 7E0030                                jmp    barrido
200
201 00F3 8604    write4    ldaa    #4                                ;se guarda el valor presionado en el stack
202 00F5 36                                psha                                ;para ser recibido por la subrutina
203 00F6 BD0000                                jsr    show                                ;que lo muestra en el display.
204 00F9 7E0030                                jmp    barrido                                ;vuelve a leer
205
206 00FC 8605    write5    ldaa    #5
207 00FE 36                                psha
208 00FF BD0000                                jsr    show
209 0102 7E0030                                jmp    barrido
210
211 0105 8606    write6    ldaa    #6
212 0107 36                                psha
213 0108 BD0000                                jsr    show
214 010B 7E0030                                jmp    barrido
215
216 010E 8607    write7    ldaa    #7
217 0110 36                                psha
218 0111 BD0000                                jsr    show
219 0114 7E0030                                jmp    barrido
220
221 0117 8608    write8    ldaa    #8
222 0119 36                                psha
223 011A BD0000                                jsr    show
224 011D 7E0030                                jmp    barrido
225
226 0120 8609    write9    ldaa    #9

```

```

227 0122 36          psha
228 0123 BD0000      jsr      show
229 0126 7E0030      jmp      barrido
230
231
232
233 *****
234 *          SECCION DE OPERACIONES          *
235 *                                          *
236 *  Esta parte del programa se encarga    *
237 *  de realizar las operaciones          *
238 *  indicadas mediante el uso            *
239 *  del teclado                          *
240 *****
241
242 0129              rseg      CODE
243
244 0129 B60000      wsuma      ldaa      counter      ;se fija si ya se ingresó la primer cifra
                                           ;del número
245 012C 8102        cmpa      #2
246 012E 2708        beq      endsuma      ;si no se ingresó no hace nada
247
248 0130 862B        ldaa      #$2B        ;si se ingresó guarda el operando en el
                                           ;stack
249 0132 36          psha
250 0133 8601        ldaa      #1          ;y enciende el flag de suma para que se sepa
251 0135 B70001      staa      flags        ;la operacion a realizar cuando se presiona
                                           ;el igual
252
253 0138 7E0018      endsuma     jmp      clear      ;salta a leer el principio de un numero
254
255
256 013B B60000      wresta      ldaa      counter      ;se fija si ya se ingresó la primer cifra
                                           ;del número
257 013E 8102        cmpa      #2
258 0140 2629        bne      endresta      ;si ya se ingresó pasa a registrar el
                                           ;operando
259
260 0142 C603        ldab      #DI_START      ;si no se ingresó imprime un signo menos al
261 0144 862D        ldaa      #$2D          ;principio del display
262 0146 BD002A      jsr      outchar
263
264 0149 8601        ldaa      #1
265 014B B10001      cmpa      flags
266 014E 2715        beq      setflagn2
267 0150 B10002      cmpa      flagr        ;se fija si trabaja con el segundo número o
                                           ;el primero
268 0153 2710        beq      setflagn2      ;si es con el segundo prende el flag que
269 0155 B10003      cmpa      flagm        ;indica que el segundo número es negativo
270 0158 270B        beq      setflagn2
271 015A B10004      cmpa      flagd
272 015D 2706        beq      setflagn2
273
274 015F B70005      staa      flagn1        ;si es con el primero prende el flag que
275 0162 7E0030      jmp      barrido      ;indica que el primer número es negativo y
                                           ;vuelve a leer
276
277 0165 B70006      setflagn2     staa      flagn2
278 0168 7E0030      jmp      barrido      ;vuelve a leer
279
280 016B 8601        endresta     ldaa      #1
281 016D B70002      staa      flagr        ;indica que la operación elegida fue la
                                           ;resta
282 0170 862D        ldaa      #$2D        ;guarda el signo menos en el stack
283 0172 36          psha

```

```

284 0173 7E0018      jmp      clear
285
286
287 0176 B60000      wmulti      ldaa      counter      ;se fija si ya se ingresó la primer cifra
                                         ;del número
288 0179 8102          cmpa      #2
289 017B 2708          beq      endmulti      ;si no se ingresó no hace nada
290
291 017D 862A          ldaa      #$2A          ;si se ingresó guarda el operando en el
                                         ;stack
292 017F 36            psha
293 0180 8601          ldaa      #1          ;y enciende el flag de multiplicación para
294 0182 B70003          staa      flagm      ;que se sepa la operación a realizar cuando
                                         ;se presiona el igual
295
296 0185 7E0018      endmulti      jmp      clear      ;salta a leer el principio de un numero
297
298
299 0188 B60000      wdivision      ldaa      counter      ;se fija si ya se ingresó la primer cifra
                                         ;del número
300 018B 8102          cmpa      #2
301 018D 2708          beq      enddivision      ;si no se ingresó no hace nada
302
303 018F 862F          ldaa      #$2F          ;si se ingresó guarda el operando en el
                                         ;stack
304 0191 36            psha
305 0192 8601          ldaa      #1          ;y enciende el flag de división para que se
306 0194 B70004          staa      flagd      ;sepa la operación a realizar cuando se
                                         ;presiona el igual
307
308 0197 7E0018      enddivision      jmp      clear      ;salta a leer el principio de un numero
309
310
311 019A C603          wigual      ldab      #DI_START      ;toma el valor de la primera posición del
                                         ;display
312 019C CE0004          ldx      #4          ;inicia contador de posiciones del display
313 019F 8600          ldaa      #0          ;toma el valor 0 para borrar el display
314 01A1 BD002A      cleanigual      jsr      outchar      ;llama a la subrutina para poner los valores
                                         ;del display
315 01A4 5A          decb
316 01A5 09          dex
317 01A6 26F9          bne      cleanigual      ;decrementa la posición del display
                                         ;decrementa el contador de posiciones
                                         ;si el contador no es 0 repite la secuencia
318
319 01A8 8601          ldaa      #1
320 01AA B10001          cmpa      flags
321 01AD 270F          beq      dosuma
322 01AF B10002          cmpa      flagr
323 01B2 2710          beq      doresta      ;se fija que operación fue elegida
324 01B4 B10003          cmpa      flagm
325 01B7 2711          beq      domulti
326 01B9 B10004          cmpa      flagd
327 01BC 2712          beq      dodivision
328
329
330
331 01BE BD0000      dosuma      jsr      sumar
332 01C1 7E0030      jmp      barrido
333 01C4 BD0000      doresta      jsr      restar
334 01C7 7E0030      jmp      barrido      ;salta a la subrutina de la operación
335 01CA BD0000      domulti      jsr      multi
336 01CD 7E0030      jmp      barrido
337 01D0 BD0000      dodivision      jsr      division
338 01D3 7E0030      jmp      barrido
339
340

```

```

341
342 *****
343 *          SECCION DE SUBROUTINAS          *
344 *                                          *
345 *  Aquí se encuantran las subrutinas      *
346 *  de delay, de inicializacion del      *
347 *  display y de envio de caracteres al  *
348 *          display                      *
349 *****
350
351 0000      delay      rseg      SUBR
352
353 0000              org      *
354 0000 36          psha
355 0001 37          pshb          ;se hace un backup de los registros a
356 0002 3C          pshx          ;modificas
357
358 0003 CE0003      ldx      #L          ;crea el contador mayor
359 0006 09      countl      dex
360 0007 270D      beq      endelay          ;si es cero el contador mayor va a terminar
                                          ;el programa
361
362 0009 86FF              ldaa      #M          ;si no es cero reinicia el contador mediano
363 000B 4A      countm      decb
364 000C 27F8      beq      countl          ;si es cero el contador mediano pasa a
                                          ;decrementar el mayor
365
366 000E C6FF              ldab      #S          ;si no es cero reinicia el contador menor
367 0010 5A      counts      decb
368 0011 26FD      bne      counts
369 0013 7E000B      jmp      countm          ;si es cero el contador menor decrementa el
                                          ;mediano
370
371 0016 38      endelay      pulx
372 0017 33          pulb          ;se recuperan los valores guardados
373 0018 32          pula
374 0019 39          rts
375
376
377
378 001A              rseg      SUBR
379
380 001A      initdis      EQU      *
381
382 001A 36          psha
383
384 001B 860F              ldaa      #$0F
385 001D B71007          staa      DDRC          ;se prepara el display
386 0020 B61002          ldaa      PIOC
387 0023 84FE          anda      #$FE
388 0025 B71002          staa      PIOC
389
390 0028 32          pula
391 0029 39          rts
392
393 002A      outchar      EQU      *
394
395 002A F71005          stab      PORTCL          ;se envían los caracteres
396 002D B71004          staa      PORTB
397
398 0030 39          rts
399
400
401
402

```



```

403 *****
404 *          AREA DE VARIABLES          *
405 *                                     *
406 *   Aquí se guardan las posiciones de *
407 *   memoria que serán usadas como   *
408 *   variables                         *
409 *                                     *
410 *****
411
412 0000                rseg    VARS
413
414 0000    counter      rmb     1           ;este contador se guarda la cantidad de
                                           ;dígitos
415
416 0001    flags        rmb     1
417 0002    flagr        rmb     1           ;estos flags guardan la operación elegida
418 0003    flagm        rmb     1
419 0004    flagd        rmb     1
420
421 0005    flagn1        rmb     1           ;estos flags guardan el signo de los números
422 0006    flagn2        rmb     1
423
424 0007 00            fcb     0
425 0008    msg          rmb     2           ;esto se usa para retener los dígitos de los
                                           ;números
426
427
428
429
430 *****
431 *          AREA DEL STACK            *
432 *                                     *
433 *   Aquí se guardan las posiciones de *
434 *   memoria que serán usadas como   *
435 *   stack                           *
436 *                                     *
437 *****
438
439 0000                rseg    STACK
440 0000    stack        rmb    100
441 0063    stackP       EQU    *-1
442
443
444
445 0064                END

Errors:  None          #####
Bytes:   520          # tp6j #
CRC:     FFB6         #####

```

SHOWNUM

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           28/Oct/20  11:21:15  #
#
#   Source   =   shownum.msa                        #
#   List     =   shownum.lst                        #
#   Object   =   shownum.r07                        #
#   Options  =                                     #
#
#                                           (c) Copyright IAR Systems 1990 #
#####
```

```

1  0000                                p68h11
2
3
4
5  0000                                extern  outchar,counter,msg,error
6  0000                                public  show
7
8  0003                                DI_START    EQU    $03
9  0000                                DI_END      EQU    $00
10
11
12
13 0000                                rseg      SUBR
14
15 0000 C600      show                ldab      #DI_END      ;toma el valor de la primera posición del
                                           ;display
16 0002 CE0003                                ldz      #3          ;inicia contador de posiciones del display
17 0005 8600                                ldaa     #0          ;toma el valor 0 para borrar el display
18 0007 BD0000      cleanshow          jsr      outchar      ;llama a la subrutina para poner los valores
                                           ;del display
19 000A 5C                                incb                                ;decrementa la posición del display
20 000B 09                                dex                                ;decrementa el contador de posiciones
21 000C 26F9                                bne      cleanshow    ;si el contador no es 0 repite la secuencia
22
23 000E 18CE0000                                ldy      #msg        ;se apunta IY a donde se guardaran los
                                           ;caracteres
24 0012 30                                tsx                                ;se crea un frame pointer
25 0013 A602                                ldaa     2,x          ;se carga a con el ultimo digito tecleado
26 0015 C630                                ldab     #$30         ;se lo convierte a ASCII
27 0017 1B                                aba
28 0018 18A701                                staa     1,y          ;se retiene al carácter en una posición de
                                           ;memoria
29 001B 7A0000                                dec      counter      ;se actualiza la cantidad de dígitos
                                           ;ingresados
30 001E 2B1A                                bmi      showerror    ;si se presionan tres dígitos seguidos,
                                           ;error
31 0020 2608                                bne      undigit      ;si el número es de un digito pasa a
                                           ;imprimirlo
32
33 0022 A603                                ldaa     3,x          ;si no carga el otro digito
34 0024 C630                                ldab     #$30         ;se lo convierte a ASCII
35 0026 1B                                aba
36 0027 18A700                                staa     0,y          ;se retiene el carácter

```

```

37
38 002A C600      undigit      ldab      #DI_END
39
40 002C 18A601    dosdig       ldaa      1,y
41 002F 2708      beq          endshow      ;se imprime el numero entero hasta ahora, ya
42 0031 BD0000    jsr          outchar      ;sea de una o dos cifras
43 0034 5C        incb
44 0035 1809      dey
45 0037 20F3      bra          dosdig
46 0039 39        endshow      rts          ;vuelve a barrer
47
48 003A BD0000    showerror     jsr          error
49 003D 7E0039    jmp          endshow
50
51 0040                                END

```

```

Errors:  None      #####
Bytes:   64        # shownum #
CRC:     D9B5      #####

```

ERROR

```

#####
#
#   Micro Series 6801 Assembler V2.00/DOS           25/Oct/20  12:41:54  #
#
#   Source   =   error.msa                          #
#   List     =   error.lst                          #
#   Object   =   error.r07                          #
#   Options  =                                     #
#
#                                                     (c) Copyright IAR Systems 1990 #
#####

```

```

1 0000                                p68h11
2
3
4
5 0000                                public error
6 0000                                extern initdis,outchar
7
8 0045      MISTAKE      EQU      $45
9
10 0003      DI_START    EQU      $03
11 0000      DI_END      EQU      $00
12
13
14
15 0000                                rseg      SUBR
16
17 0000 36      error     psha                                ;hace backup de los registros
18 0001 37      pshb
19 0002 3C      pshx
20 0003 183C    pshy
21
22 0005 C603      ldab      #DI_START      ;toma el valor de la primera posición del
                                           ;display
23 0007 CE0004    ldx       #4              ;inicia contador de posiciones del display
24 000A 8600      preperror ldaa      #0      ;toma el valor 0 para borrar el display
25 000C BD0000    jsr          outchar      ;llama a la subrutina para poner los valores
                                           ;del display

```

```

26 000F 5A          decb          ;decrementa la posición del display
27 0010 09          dex          ;decrementa el contador de posiciones
28 0011 26F7        bne          preperror ;si el contador no es 0 repite la secuencia
29
30
31 0013 C603        ldab          #DI_START ;toma el valor de la primera posición del
                                   ;display
32 0015 8645        ldaa          #MISTAKE ;toma la letra E
33 0017 BD0000      jsr          outchar  ;la manda al display
34
35 001A 1838        puly          ;hace restore de los registros
36 001C 38          pulx
37 001D 33          pulb
38 001E 32          pula
39
40 001F 39          rts            ;vuelve al programa principal
41
42
43
44 0020            END

```

```

Errors:  None          #####
Bytes:   32           # error #
CRC:     819A         #####

```

SUMAC

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           28/Oct/20   10:43:05   #
#
#   Source   =   sumac.msa                               #
#   List     =   sumac.lst                               #
#   Object   =   sumac.r07                               #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####
```

```

1  0000                                p68h11
2
3  0000                                extern  stackP,outchar,flagn1,flagn2,error
4  0000                                public  sumar
5
6  0003      DI_START      EQU      $03
7  0000      DI_END       EQU      $00
8  0045      VARIAN       EQU      $45
9  0001      FINSTACK     EQU      stackP+1
10
11
12      *****
13      *   SUBROUTINA suma:                                *
14      *   esta subrutina toma los números ingresados por el *
15      *   usuario por stack, suma los números y manda el   *
16      *   resultado al display                               *
17      *****
18
19  0000      sumar        rseg      SUBR
20
21  0000 36                                psha
22  0001 36                                psha                                ;hago espacio en el stack para guardar los
23  0002 37                                pshb                                ;números a utilizar y los ASCII del
                                           ;resultado
24  0003 1830                                tsy
25  0005 3C                                pshx                                ;hace backup de los registros
26  0006 183C                                pshy
27  0008 8645                                ldaa      #VARIAN
28  000A B70001                        staa      FINSTACK                ;carga una posición anterior al comienzo del
                                           ;stack un número arbitrario
29
30
31
32      *****
33      *   SECCIÓN DE RECIBO DE NÚMEROS                      *
34      *   esta sección se encarga de tomar los números     *
35      *   ingresados usuario por stack                      *
36      *****
37
38  000D 18E606                        ldab      6,y                    ;analiza el segundo número ingresado
39  0010 C12B                        cmpb      #$2B                    ;ve si es un operando o un dígito

```

40	0012 270D		beq	lop	;si es el operando, el segundo número es de
41	0014 860A		ldaa	#\$A	;un dígito
42	0016 3D		mul		;si es un dígito, el segundo número es de
43	0017 18A605		ldaa	5,y	;dos dígitos
44	001A 1B		aba		;multiplica el dígito por 10, lo transforma
45	001B 18A701		staa	1,y	;a decenas
46	001E 7E002A		jmp	lop1	;toma las unidades del segundo número
47					;las suma a las decenas
48	0021 18A605	lop	ldaa	5,y	;guarda el segundo número en hexadecimal en
49	0024 18A701		staa	1,y	;el stack
50	0027 7E003E		jmp	lop2	;analiza el primer número ingresado
51					
52	002A 18E609	lop1	ldab	9,y	;toma el único dígito del segundo número
53	002D C145		cmpb	#VARIAN	;guarda el segundo número en hexadecimal en
54	002F 2721		beq	lop3	;el stack
55	0031 860A		ldaa	#\$A	;analiza el primer número ingresado
56	0033 3D		mul		
57	0034 18A608		ldaa	8,y	;toma el valor 9 lugares después del stack
58	0037 1B		aba		; lo compara con un valor
59	0038 18A702		staa	2,y	;si es ese valor, el primer número tiene un
60	003B 7E0064		jmp	negati	;dígito
61					;si no es ese valor, el primer número tiene
62	003E 18E608	lop2	ldab	8,y	;dos dígitos
63	0041 C145		cmpb	#VARIAN	;multiplica el dígito por 10, lo transforma
64	0043 2716		beq	lop4	;a decenas
65	0045 860A		ldaa	#\$A	;toma las unidades del primer número
66	0047 3D		mul		;las suma a las decenas
67	0048 18A607		ldaa	7,y	;guarda el primer número en hexadecimal en
68	004B 1B		aba		;el stack
69	004C 18A702		staa	2,y	;analiza si son negativos
70	004F 7E0064		jmp	negati	
71					
72	0052 18E608	lop3	ldab	8,y	;toma el único dígito del primer número
73	0055 18E702		stab	2,y	;guarda el primer número en hexadecimal en
74	0058 7E0064		jmp	negati	;el stack
75	005B 18E607	lop4	ldab	7,y	;analiza si son negativos
76	005E 18E702		stab	2,y	;toma el único dígito del primer número
77	0061 7E0064		jmp	negati	;guarda el primer número en hexadecimal en
78					;el stack
79					;analiza si son negativos
80					
81	0064 B60000	negati	ldaa	flagn1	;toma el valor del indicador de negatividad
82	0067 8101		cmpa	#1	;del primer número
83	0069 2702		beq	comple1	;el primer número es negativo, hace el
84	006B 2009		bra	negati2	;complemento a 2
85					;el primer número es positivo, analizo el
86					;segundo

```

87
88 006D 18A602 comple1 ldaa 2,y ;toma el valor del primer número
89 0070 40 nega ;hace el complemento a 2
90 0071 18A702 staa 2,y ;lo guarda nuevamente
91 0074 2000 bra negati2 ;analizo el segundo
92
93 0076 B60000 negati2 ldaa flagn2 ;toma el valor del indicador de negatividad
;del segundo número
94 0079 8101 cmpa #1
95 007B 2702 beq serror ;el segundo número es negativo, error
96 007D 200A bra opera ;el segundo número es positivo, hace la
;operación
97
98 007F 1838 serror pulv ;restore de los backups
99 0081 38 pulx
100 0082 33 pulb
101 0083 32 pula
102 0084 32 pula
103 0085 BD0000 jsr error ;manda E al display
104 0088 39 rts
105
106
107
108 *****
109 * SECCIÓN DE OPERACIÓN *
110 * esta sección se encarga de sumar los números *
111 * recibiendo los por stack y devuelve el resultado por *
112 * stack *
113 *****
114
115 0089 opera EQU *
116 0089 18A602 ldaa 2,y ;toma el valor del primer número
117 008C 18E601 ldab 1,y ;toma el valor del segundo número
118 008F 1B aba ;los suma
119 0090 2902 bvs flow
120 0092 2B06 bmi sign ;si el resultado es negativo, hace el
;complemento a 2
121 0094 18A700 flow staa 0,y ;si no es negativo el resultado, lo guarda
122 0097 7E00A8 jmp disp2
123
124
125 009A 40 sign nega ;hace el complemento a 2 del resultado
126 009B 18A700 staa 0,y ;lo guarda nuevamente
127 009E C603 ldab #DI_START ;toma el valor de la primera posición del
;display
128 00A0 862D ldaa #$2D ;toma el valor ASCII del signo menos
129 00A2 BD0000 jsr outchar ;lo manda al display
130 00A5 7E00A8 jmp disp2
131
132
133
134 *****
135 * SECCIÓN DE TRANSFORMACIÓN A ASCII Y DISPLAY *
136 * esta sección se encarga de transformar el resultado *
137 * en ASCII y mandarlo al display *
138 *****
139
140 00A8 18A600 disp2 ldaa 0,y ;toma el resultado de la suma
141 00AB 810A cmpa #$A ;ve si el número se encuentra entre 0-9
142 00AD 252A blo lop5 ;si se encuentra entre 0-9, pasa a ASCII ese
;dígito
143 00AF 8163 cmpa #99 ;ve si el número es mayor a 100
144 00B1 223A bhi lop6
145 00B3 4F clra

```

146	00B4 18E600		ldab	0,y	;si es mayor a 9 y menor a 100, toma ;nuevamente el resultado
147	00B7 CE000A		ldx	#\$A	
148	00BA 02		idiv		;divide el resultado por 10
149	00BB CDEF00		stx	0,y	;guarda el primer dígito del resultado
150	00BE 18E700		stab	0,y	;guarda el segundo dígito del resultado
151	00C1 8600		ldaa	#0	
152	00C3 18A702		staa	2,y	;pone en 0 la posición de centenas
153					
154	00C6 18A601		ldaa	1,y	;toma el primer dígito del resultado
155	00C9 C630		ldab	#\$30	
156	00CB 1B		aba		;lo convierte en ASCII
157	00CC 18A701		staa	1,y	;guarda el primer dígito en ASCII (decenas)
158	00CF 18A600		ldaa	0,y	;toma el segundo dígito del resultado
159	00D2 1B		aba		;lo transforma en ASCII
160	00D3 18A700		staa	0,y	;guarda el segundo dígito en ASCII
161	00D6 7E011C		jmp	disp1	
162					
163	00D9 18A600	lop5	ldaa	0,y	;toma el resultado
164	00DC C630		ldab	#\$30	
165	00DE 1B		aba		;lo transgorma en ASCII
166	00DF 18A700		staa	0,y	;lo guarda nuevamente (unidades)
167	00E2 8600		ldaa	#0	
168	00E4 18A701		staa	1,y	;pone en 0 la posición de decenas
169	00E7 18A702		staa	2,y	;pone en 0 la posición de centenas
170	00EA 7E011C		jmp	disp1	
171					
172	00ED 4F	lop6	clra		
173	00EE 18E600		ldab	0,y	;si es mayor a 100, toma el resultado
174	00F1 CE0064		ldx	#\$64	
175	00F4 02		idiv		;divide el resultado por 100
176	00F5 CDEF01		stx	1,y	;guarda el primer dígito del resultado
177	00F8 CE000A		ldx	#\$A	
178	00FB 02		idiv		;divido el resto por 10
179	00FC CDEF00		stx	0,y	;guarda del segundo dígito del resultado
180	00FF 18E700		stab	0,y	;guarda el tercer dígito del resultado
181					
182	0102 18A602		ldaa	2,y	;toma el primer dígito del resultado
183	0105 C630		ldab	#\$30	
184	0107 1B		aba		;lo transforma a ASCII
185	0108 18A702		staa	2,y	;guarda el primer dígito en ASCII
186	010B 18A601		ldaa	1,y	;toma el segundo dígito del resultado
187	010E 1B		aba		;lo transforma a ASCII
188	010F 18A701		staa	1,y	;guarda el segundo dígito en ASCII
189	0112 18A600		ldaa	0,y	;toma el tercer dígito del resultado
190	0115 1B		aba		;lo tranforma a ASCII
191	0116 18A700		staa	0,y	;guarda el tercer dígito en ASCII
192	0119 7E011C		jmp	disp1	
193					
194					
195					
196					
197					
198	011C C600	disp1	ldab	#DI_END	;toma el valor de la última posición del ;display
199	011E CE0003		ldx	#3	;inicia contador de números a mostrar
200	0121 18A600	repito	ldaa	0,y	;toma el dígito en ASCII
201	0124 BD0000		jsr	outchar	;lo manda al display
202	0127 09		dex		;decremento el contador
203	0128 2705		beq	theen	;si el contador es 0, termino la subrutina
204	012A 5C		incb		;si no es cero el contador, toma el valor de ;la anterior posición del display
205	012B 1808		iny		;apunta al siguiente dígito
206	012D 20F2		bra	repito	;repite proceso
207					

Introducción a la Computación
TP6 Calculadora

Grupo 4

```
208 012F 1838      then      puly
209 0131 38                pulx
210 0132 33                pulb
211 0133 32                pula
212 0134 32                pula
213 0135 39                rts
214
215
216
217 0136                END
```

```
Errors:  None      #####
Bytes:   310       # sumac #
CRC:     4666      #####
```

RESTAC

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS      28/Oct/20  10:43:13  #
#
#   Source   =  restac.msa
#   List     =  restac.lst
#   Object   =  restac.r07
#   Options  =
#
#
#                                     (c) Copyright IAR Systems 1990 #
#####
```

```
1  0000                p68h11
2
3  0000                extern  stackP,outchar,flagn1,flagn2,error
4  0000                public  restar
5
6  0003                DI_START      EQU    $03
7  0000                DI_END        EQU    $00
8  0045                VARIAN        EQU    $45
9  0001                FINSTACK      EQU    stackP+1
10
11
12                *****
13                *   SUBROUTINA  restar:
14                *   esta subrutina toma los números ingresados por el
15                *   usuario por stack, resta los números y manda el
16                *   resultado al display
17                *****
18
19 0000                rseg    SUBR
20
21 0000 36            restar    psha
```

```

22 0001 36          psha          ;hago espacio en el stack para guardar los
23 0002 37          pshb          ;números a utilizar y los ASCII del
                                   ;resultado

24 0003 1830        tsy
25 0005 3C          pshx          ;hace backup de los registros
26 0006 183C        pshy
27 0008 8645        ldaa    #VARIAN
28 000A B70001       staa    FINSTACK ;carga una posición anterior al comienzo del
                                   ;stack un número arbitrario

29
30
31
32 *****
33 *   SECCIÓN DE RECIBO DE NÚMEROS   *
34 *   esta sección se encarga de tomar los números   *
35 *   ingresados usuario por stack   *
36 *****
37
38 000D 18E606       ldab    6,y    ;analiza el segundo número ingresado
39 0010 C12D         cmpb    #$2D   ;ve si es un operando o un dígito
40 0012 270D         beq     lp      ;si es el operando, el segundo número es de
                                   ;un dígito
41 0014 860A        ldaa    #$A     ;si es un dígito, el segundo número es de
                                   ;dos dígitos
42 0016 3D          mul           ;multiplica el dígito por 10, lo transforma
                                   ;a decenas
43 0017 18A605       ldaa    5,y    ;toma las unidades del segundo número
44 001A 1B          aba           ;las suma a las decenas
45 001B 18A701       staa    1,y    ;guarda el segundo número en hexadecimal en
                                   ;el stack
46 001E 7E002A       jmp     lp1     ;analiza el primer número ingresado
47
48 0021 18A605       ldaa    5,y    ;toma el único dígito del segundo número
49 0024 18A701       staa    1,y    ;guarda el segundo número en hexadecimal en
                                   ;el stack
50 0027 7E003E       jmp     lp2     ;analiza el primer número ingresado
51
52 002A 18E609       ldab    9,y    ;toma el valor 9 lugares después del stack
53 002D C145         cmpb    #VARIAN ; lo compara con un valor
54 002F 2721         beq     lp3     ;si es ese valor, el primer número tiene un
                                   ;dígito
55 0031 860A        ldaa    #$A     ;si no es ese valor, el primer número tiene
                                   ;dos dígitos
56 0033 3D          mul           ;multiplica el dígito por 10, lo transforma
                                   ;a decenas
57 0034 18A608       ldaa    8,y    ;toma las unidades del primer número
58 0037 1B          aba           ;las suma a las decenas
59 0038 18A702       staa    2,y    ;guarda el primer número en hexadecimal en
                                   ;el stack
60 003B 7E0064       jmp     negat   ;analiza si son negativos
61
62 003E 18E608       ldab    8,y    ;toma el valor 8 lugares después del stack
63 0041 C145         cmpb    #VARIAN ; lo compara con un valor
64 0043 2716         beq     lp4     ;si es ese valor, el primer número tiene un
                                   ;dígito
65 0045 860A        ldaa    #$A     ;si no es ese valor, el primer número tiene
                                   ;dos dígitos
66 0047 3D          mul           ;multiplica el dígito por 10, lo transforma
                                   ;a decenas
67 0048 18A607       ldaa    7,y    ;toma las unidades del primer número
68 004B 1B          aba           ;las suma a las decenas
69 004C 18A702       staa    2,y    ;guarda el primer número en hexadecimal en
                                   ;el stack
70 004F 7E0064       jmp     negat   ;analiza si son negativos
71

```

```

72 0052 18E608    lp3      ldab    8,y      ;toma el único dígito del primer número
73 0055 18E702      stab    2,y      ;guarda el primer número en hexadecimal en
                                   ;el stack
74 0058 7E0064      jmp     negat    ;analiza si son negativos
75 005B 18E607    lp4      ldab    7,y      ;toma el único dígito del primer número
76 005E 18E702      stab    2,y      ;guarda el primer número en hexadecimal en
                                   ;el stack
77 0061 7E0064      jmp     negat    ;analiza si son negativos
78
79
80
81 0064 B60000      negat    ldaa    flagn1   ;toma el valor del indicador de negatividad
                                   ;del primer número
82 0067 8101      cmpa    #1
83 0069 2702      beq     compl1   ;el primer número es negativo, hace el
                                   ;complemento a 2
84 006B 2009      bra     negat2   ;el primer número es positivo, analizo el
                                   ;segundo
85
86
87
88 006D 18A602      compl1   ldaa    2,y      ;toma el valor del primer número
89 0070 40        nega      ;hace el complemento a 2
90 0071 18A702      staa    2,y      ;lo guarda nuevamente
91 0074 2000      bra     negat2   ;analizo el segundo
92
93 0076 B60000      negat2   ldaa    flagn2   ;toma el valor del indicador de negatividad
                                   ;del segundo número
94 0079 8101      cmpa    #1
95 007B 2702      beq     sherror   ;el segundo número es negativo, hace error
96 007D 200A      bra     ope      ;el segundo número es positivo, hace la
                                   ;operación
97
98 007F 1838      sherror    puly      ;hace restore de los registros
99 0081 38        pulx
100 0082 33        pulb
101 0083 32        pula
102 0084 32        pula
103 0085 BD0000      jsr     error    ;manda E al display
104 0088 39        rts
105
106
107 *****
108 *   SECCIÓN DE OPERACIÓN   *
109 *   esta sección se encarga de restar los números   *
110 *   recibiendo los por stack y devuelve el resultado por *
111 *   stack   *
112 *****
113
114 0089      ope      equ     *
115 0089 18A602      ldaa    2,y      ;toma el valor del primer número
116 008C 18E601      ldab    1,y      ;toma el valor del segundo número
117 008F 50        negb
118 0090 1B        aba      ;los suma
119 0091 2908      bvs     sig
120 0093 2B06      bmi     sig      ;si el resultado es negativo, hace el
                                   ;complemento a 2
121 0095 18A700      staa    0,y      ;si no es negativo el resultado, lo guarda
122 0098 7E00A9      jmp     dis2
123
124 009B 40        sig      nega      ;hace el complemento a 2 del resultado
125 009C 18A700      staa    0,y      ;lo guarda nuevamente
126 009F C603      ldab    #DI_START ;toma el valor de la primera posición del
                                   ;display
127 00A1 862D      ldaa    #$2D     ;toma el valor ASCII del signo menos

```

```

128 00A3 BD0000      jsr     outchar      ;lo manda al display
129 00A6 7E00A9      jmp     dis2
130
131
132
133
134 *****
135 *   SECCIÓN DE TRANSFORMACIÓN A ASCII Y DISPLAY   *
136 *   esta sección se encarga de transformar el resultado *
137 *   en ASCII y mandarlo al display                 *
138 *****
139
140 00A9 18A600      dis2      ldaa     0,y      ;toma el resultado de la suma
141 00AC 810A        cmpa     #$A      ;ve si el número se encuentra entre 0-9
142 00AE 252A        blo      lp5      ;si se encuentra entre 0-9, pasa a ASCII ese
                                      ;dígito
143 00B0 8163        cmpa     #99      ;ve si el número es mayor a 100
144 00B2 223A        bhi      lp6
145 00B4 4F          clra
146 00B5 18E600      ldab     0,y      ;si es mayor a 9 y menor a 100, toma
                                      ;nuevamente el resultado
147 00B8 CE000A      ldx      #$A
148 00BB 02          idiv
                                      ;divide el resultado por 10
149 00BC CDEF00      stx      0,y      ;guarda el primer dígito del resultado
150 00BF 18E700      stab     0,y      ;guarda el segundo dígito del resultado
151 00C2 8600        ldaa     #0
152 00C4 18A702      staa     2,y      ;pone en 0 la posición de centenas
153
154 00C7 18A601      ldaa     1,y      ;toma el primer dígito del resultado
155 00CA C630        ldab     #$30
156 00CC 1B          aba
                                      ;lo convierte en ASCII
157 00CD 18A701      staa     1,y      ;guarda el primer dígito en ASCII (decenas)
158 00D0 18A600      ldaa     0,y      ;toma el segundo dígito del resultado
159 00D3 1B          aba
                                      ;lo transforma en ASCII
160 00D4 18A700      staa     0,y      ;guarda el segundo dígito en ASCII
161 00D7 7E011D      jmp     dis1
162
163 00DA 18A600      lp5      ldaa     0,y      ;toma el resultado
164 00DD C630        ldab     #$30
165 00DF 1B          aba
                                      ;lo transgorma en ASCII
166 00E0 18A700      staa     0,y      ;lo guarda nuevamente (unidades)
167 00E3 8600        ldaa     #0
168 00E5 18A701      staa     1,y      ;pone en 0 la posición de decenas
169 00E8 18A702      staa     2,y      ;pone en 0 la posición de centenas
170 00EB 7E011D      jmp     dis1
171
172 00EE 4F          lp6      clra
173 00EF 18E600      ldab     0,y      ;si es mayor a 100, toma el resultado
174 00F2 CE0064      ldx      #$64
175 00F5 02          idiv
                                      ;divide el resultado por 100
176 00F6 CDEF01      stx      1,y      ;guarda el primer dígito del resultado
177 00F9 CE000A      ldx      #$A
178 00FC 02          idiv
                                      ;divido el resto por 10
179 00FD CDEF00      stx      0,y      ;guarda del segundo dígito del resultado
180 0100 18E700      stab     0,y      ;guarda el tercer dígito del resultado
181
182 0103 18A602      ldaa     2,y      ;toma el primer dígito del resultado
183 0106 C630        ldab     #$30
184 0108 1B          aba
                                      ;lo transforma a ASCII
185 0109 18A702      staa     2,y      ;guarda el primer dígito en ASCII
186 010C 18A601      ldaa     1,y      ;toma el segundo dígito del resultado
187 010F 1B          aba
                                      ;lo transforma a ASCII
188 0110 18A701      staa     1,y      ;guarda el segundo dígito en ASCII
189 0113 18A600      ldaa     0,y      ;toma el tercer dígito del resultado
190 0116 1B          aba
                                      ;lo tranforma a ASCII

```

```

191 0117 18A700      staa    0,y          ;guarda el tercer dígito en ASCII
192 011A 7E011D      jmp     dis1
193
194
195
196
197 011D C600      dis1      ldab    #DI_END      ;toma el valor de la última posición del
                                   ;display
198 011F CE0003      ldax    #3              ;inicia contador de números a mostrar
199 0122 18A600      repit    ldaa    0,y          ;toma el dígito en ASCII
200 0125 BD0000      jsr     outchar        ;lo manda al display
201 0128 09          dex          ;decremento el contador
202 0129 2705      beq     thee          ;si el contador es 0, termino la subrutina
203 012B 5C          incb          ;si no es cero el contador, toma el valor de
                                   ;la anterior posición del display
204 012C 1808          iny          ;apunta al siguiente dígito
205 012E 20F2      bra     repit        ;repite proceso
206
207 0130 1838      thee      puly
208 0132 38          pulx
209 0133 33          pulb
210 0134 32          pula
211 0135 32          pula
212 0136 39          rts
213
214 0137          END

Errors:  None          #####
Bytes:   311          # restac #
CRC:     1719         #####

```

PRODU

```

#####
#
#   Micro Series 6801 Assembler V2.00/DOS          26/Oct/20   18:22:15   #
#
#   Source   =   producto.msa                      #
#   List     =   producto.lst                      #
#   Object   =   producto.r07                     #
#   Options  =                                     #
#
#                                                    (c) Copyright IAR Systems 1990 #
#####

```

```

1  0000          p68H11
2
3  0000          extern  stackP,outchar,flagn1,flagn2,error
4  0000          public  multi
5
6                *****
7                *      DEFINICIONES      *
8                *****
9
10 000A          DECENAS      EQU    $0A
11 0003          LIM_HIGH    EQU    $03

```

```

12 00E7          LIM_LOW          EQU    $E7
13 0064          ASCII_HIGH       EQU    $64
14 0045          TERM             EQU    $45
15 0001          ENDSTACK         EQU    stackP+1
16 0003          DI_START        EQU    $03
17 0000          DI_END          EQU    $00
18
19              *****
20              *          PRODUCTO          *
21              *****
22
23 0000          rseg             SUBR
24 0000 3C      multi            pshx                ;backup de los registros
25 0001 30                      tsx                ;crea frame pointer
26 0002 183C                      pshy
27 0004 36                      psha
28 0005 37                      pshb
29 0006 8645                      ldaa    #TERM
30 0008 B70001          staa      ENDSTACK          ;carga una posición anterior al comienzo del
                                           ;stack un número arbitrario
31
32
33              *****
34              *          SECCIÓN DE RECIBO DE NÚMEROS          *
35              *
36              *          Se encarga de recibir los números ingresados
37              *          por stack
38              *
39              *
40              *
41              *
42              *****
43
44 000B E605          ldab      5,X                ;analiza el segundo número ingresado
45 000D C12A          cmpb     #$2A                ;ve si es un operando o un dígito
46 000F 270A          beq      undigito            ;si es el operando, el segundo número es de
                                           ;un dígito
47 0011 860A          ldaa     #DECENAS            ;si es un dígito, el segundo número es de
                                           ;dos dígitos
48 0013 3D          mul                      ;multiplica el dígito por 10, lo transforma
                                           ;a decenas
49 0014 A604          ldaa     4,X                ;toma las unidades del segundo número
50 0016 1B          aba                      ;las suma a las decenas
51 0017 36          psha                      ;guarda el segundo número en hexadecimal en
                                           ;el stack
52 0018 7E0021          jmp      primernro          ;analiza el primer número ingresado
53
54 001B A604          undigito      ldaa     4,X                ;toma el único dígito del segundo número
55 001D 36          psha                      ;guarda el segundo número en hexadecimal en
                                           ;el stack
56 001E 7E0031          jmp      primernro2         ;analiza el primer número ingresado
57
58 0021 E608          primernro      ldab      8,X                ;toma el valor 8 lugares después del stack
59 0023 C145          cmpb     #TERM                ; lo compara con un valor
60 0025 271A          beq      undigito2           ;si es ese valor, el primer número tiene un
                                           ;dígito
61 0027 860A          ldaa     #DECENAS            ;si no es ese valor, el primer número tiene
                                           ;dos dígitos
62 0029 3D          mul                      ;multiplica el dígito por 10, lo transforma
                                           ;a decenas
63 002A A607          ldaa     7,X                ;toma las unidades del primer número
64 002C 1B          aba                      ;las suma a las decenas
65 002D 36          psha                      ;guarda el primer número en hexadecimal en
                                           ;el stack
66 002E 7E004A          jmp      cuenta

```

```

67
68 0031 E607      primernro2      ldab    7,X      ;toma el valor 8 lugares después del stack
69 0033 C145      cmpb    #TERM    ;lo compara con un valor
70 0035 2710      beq     undigito3 ;si es ese valor, el primer número tiene un
                                   ;dígito
71 0037 860A      ldaa    #DECENAS ;si no es ese valor, el primer número tiene
                                   ;dos dígitos
72 0039 3D        mul          ;multiplica el dígito por 10, lo transforma
                                   ;a decenas
73 003A A606      ldaa    6,X      ;toma las unidades del primer número
74 003C 1B        aba          ;las suma a las decenas
75 003D 36        psha          ;guarda el primer número en hexadecimal en
                                   ;el stack
76 003E 7E004A    jmp     cuenta
77
78 0041 E607      undigito2      ldab    7,X      ;toma el único dígito del primer número
79 0043 37        pshb          ;guarda el primer número en hexadecimal en
                                   ;el stack
80 0044 7E004A    jmp     cuenta
81
82 0047 E606      undigito3      ldab    6,X      ;toma el único dígito del primer número
83 0049 37        pshb          ;guarda el primer número en hexadecimal en
                                   ;el stack
84
85
86 *****
87 *          PRODUCTO - CONTINUACIÓN          *
88 *****
89
90 004A 33        cuenta      pulb          ;carga el primer número
91 004B 32        pula          ;carga el segundo
92 004C 3D        mul          ;multiplico los números entre sí
93 004D 37        pshb          ;guardo el resultado en el stack
94 004E 36        psha
95 004F 7E0052    jmp     chk_rta      ;verifico la validez del resultado (el valor
                                   ;absoluto debe ser menor o igual a 999 o
                                   ;$3E7)
96
97 *****
98 *          VALIDEZ DEL RESULTADO          *
99 *****
100
101 0052 32        chk_rta      pula          ;si el byte más significativo de la rta es
102 0053 8103      cmpa    #LIM_HIGH ;mayor a $03, el resultado no es válido
103 0055 2A10      bpl     no_valido ;si el byte más significativo de la rta es
                                   ;igual a $03, verifica el byte menos
                                   ;significativo
104 0057 2704      beq     chk_low      ;si el byte más significativo de la rta es
                                   ;menor a $03, el resultado es válido
105 0059 36        psha
106 005A 7E0071    jmp     valido
107
108 005D 33        chk_low      pulb          ;si el byte menos significativo de la rta es
109 005E C1E7      cmpb    #LIM_LOW  ;mayor a $E7, el resultado no es válido
110 0060 2A06      bpl     no_valido2 ;si el byte menos significativo de la rta es
                                   ;menor o igual a $E7, el resultado es
                                   ;válido
111 0062 37        pshb
112 0063 36        psha
113 0064 7E0071    jmp     valido
114
115 0067 33        no_valido     pulb          ;restore de los registros
116 0068 33        no_valido2    pulb

```

```

117 0069 32          pula
118 006A 1838        pula
119 006C 38          pulx
120 006D BD0000      jsr      error          ;salta a la subrutina error, que colocará el
                                           ;símbolo de error para mostrar en el
                                           ;display

121 0070 39          rts
122
123 *****
124 *      CONVERSIÓN A ASCII Y DISPLAY      *
125 *****
126
127 0071 32          valido      pula          ;extrae el resultado
128 0072 33          pulb
129 0073 C164        cmpb      #ASCII_HIGH      ;ve si el byte menos significativo es mayor
                                           ;a $64 (100 decimal)
130 0075 2A0B        bpl      casomayor        ;si es mayor, la rta está entre 100 y 1000
                                           ;decimal
131 0077 8B00        adda      #0              ;actualiza el estado del CCR para que
                                           ;refleje el estado del byte más
                                           ;significativo
132 0079 2607        bne      casomayor        ;si el byte más significativo no es nulo, la
                                           ;rta está entre 100 y 1000 decimal
133 007B C10A        cmpb      #DECENAS        ;sino, la rta es menor a 100 decimal. Ahora,
                                           ;ve si es mayor a 10 decimal
134 007D 2A50        bpl      casomedio        ;si es mayor, la rta está entre 10 y 100
                                           ;decimal
135 007F 7E010F      jmp      casomenor        ;si es menor, la rta está entre 0 y 10
136
137 0082 37          casomayor   pshb          ;hace un backup de la rta en hexa
138 0083 36          psha
139 0084 4F          clra          ;genera 3 espacios en el stack para guardar
                                           ;los dígitos en ascii
140 0085 36          psha
141 0086 36          psha
142 0087 36          psha
143
144 0088 183C        pshy          ;crea un FP
145 008A 1830        tsy
146
147 008C 18A605      ldaa      5,Y            ;recupera el byte más significativo de la
                                           ;rta
148 008F CE0064      ldx      #ASCII_HIGH
149 0092 02          idiv          ;divide la rta por 100 decimal
150
151 0093 3C          pshx          ;guarda el resultado en en stack
152 0094 CE000A      ldx      #DECENAS
153 0097 02          idiv          ;divide el resto por 10 decimal
154
155 0098 3C          pshx          ;guarda el resultado en el stack
156 0099 37          pshb          ;guarda el resto en el stack
157
158 009A C630        ldab      #$30
159 009C 32          pula          ;toma el dígito de las unidades
160 009D 1B          aba          ;lo convierte en ascii
161 009E 18A704      staa      4,Y            ;lo guarda en el stack
162
163 00A1 32          pula          ;toma el dígito de las decenas
164 00A2 32          pula          ;lo convierte en ascii
165 00A3 1B          aba          ;lo guarda en el stack
166 00A4 18A703      staa      3,Y
167
168 00A7 32          pula          ;toma el dígito de las centenas
169 00A8 32          pula          ;lo convierte en ascii
170 00A9 1B          aba

```


171	00AA 18A702		staa	2,Y	;lo guarda en el stack
172					
173	00AD C600		ldab	#DI_END	
174	00AF CE0003		ldx	#3	
175	00B2 18A604	escribir	ldaa	4,Y	;escribe el resultado en el display
176	00B5 BD0000		jsr	outchar	;comienza escribiendo las unidades en la
					;última
177	00B8 09		dex		;posición del display. Sigue con las decenas
					;y las
178	00B9 2705		beq	seg4_mayor	;centenas, y luego salta a "seg4_mayor" para
					;verificar
					;el símbolo del resultado.
179	00BB 5C		incb		
180	00BC 1809		dey		
181	00BE 20F2		bra	escribir	
182					
183	00C0 1838	seg4_mayor	puly		; "limpia" el stack y aumenta el valor del SP
184	00C2 1838		puly		
185	00C4 33		pulb		
186	00C5 1838		puly		
187	00C7 33		pulb		;restore de los registros
188	00C8 32		pula		
189	00C9 1838		puly		
190	00CB 38		pulx		
191	00CC 7E011F		jmp	seg4	
192					
193	00CF 37	casomedio	pshb		;hace un backup de la rta en hexa
194	00D0 36		psha		
195	00D1 4F		clra		;genera 2 espacios en el stack para guardar
					;los dígitos en ascii
196	00D2 36		psha		
197	00D3 36		psha		
198	00D4 183C		pshy		;crea un FP
199	00D6 1830		tsy		
200	00D8 18A604		ldaa	4,Y	;recupera el byte más significativo de la
					;rta
201	00DB CE000A		ldx	#DECENAS	
202	00DE 02		idiv		;divide el resto por 10 decimal
203	00DF 3C		pshx		;guarda el resultado en el stack
204	00E0 37		pshb		;guarda el resto en el stack
205	00E1 C630		ldab	#\$30	
206	00E3 32		pula		;toma el dígito de las unidades
207	00E4 1B		aba		;lo convierte en ascii
208	00E5 18A703		staa	3,Y	;lo guarda en el stack
209	00E8 32		pula		
210	00E9 32		pula		;toma el dígito de las decenas
211	00EA 1B		aba		;lo convierte en ascii
212	00EB 18A702		staa	2,Y	;lo guarda en el stack
213	00EE C600		ldab	#DI_END	
214	00F0 CE0002		ldx	#2	
215	00F3 18A603	escribir2	ldaa	3,Y	;escribe el resultado en el display
216	00F6 BD0000		jsr	outchar	;comienza escribiendo las unidades en la
					;última
217	00F9 09		dex		;posición del display. Sigue con las decenas
218	00FA 2705		beq	seg4_medio	;y luego salta a "seg4_medio" para verificar
219	00FC 5C		incb		;el símbolo del resultado.
220	00FD 1809		dey		
221	00FF 20F2		bra	escribir2	
222	0101 1838	seg4_medio	puly		; "limpia" el stack y aumenta el valor del SP
223	0103 1838		puly		
224	0105 1838		puly		
225	0107 33		pulb		;restore de los registros
226	0108 32		pula		
227	0109 1838		puly		
228	010B 38		pulx		
229	010C 7E011F		jmp	seg4	

```

230
231 010F 8630      casomenor      ldaa    #$30
232 0111 1B                aba                ;convierte la rta en ascii
233 0112 C600      ldab    #DI_END
234 0114 BD0000     jsr      outchar          ;carga el resultado en el display
235 0117 33                pulb          ;restore de los registros
236 0118 32                pula
237 0119 1838       puly
238 011B 38                pulx
239 011C 7E011F     jmp      seg4            ;verifica el símbolo del resultado
240
241                *****
242                *          SIGNO          *
243                *****
244
245 011F B60000     seg4      ldaa    flagn1
246 0122 B10000     cmpa     flagn2          ;ve si los signos de los números que se
                                           ;multiplicaron son iguales
247 0125 2707                beq      endprod      ;si son iguales, el resultado es positivo
248 0127 862D       ldaa    #$2D          ;si son distintos, el resultado es negativo.
249 0129 C603      ldab    #DI_START      ;carga el símbolo "-" al display
250 012B BD0000     jsr      outchar
251 012E 39                endprod      rts
252
253 012F                END

Errors:  None          #####
Bytes:   303          # producto #
CRC:     C495         #####

```

DIVI

```

#####
#
#   Micro Series 6801 Assembler V2.00/DOS          25/Oct/20  17:08:33  #
#
#   Source   =   divi.msa                                #
#   List     =   divi.lst                                #
#   Object   =   divi.r07                                #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

```

```

1
2 0000                p68h11
3
4 0000                extern  stackP,outchar,flagn1,flagn2
5 0000                public  division
6
7 0003                DI_START      EQU    $03
8 0000                DI_END        EQU    $00
9 0045                VARIAN        EQU    $45
10 0001                FINSTACK     EQU    stackP+1
11
12

```

```

13 *****
14 *   SUBROUTINA  division:                               *
15 *   esta subrutina toma los números ingresados por el   *
16 *   usuario por stack, divide los números y manda el    *
17 *   resultado al display                                *
18 *****
19
20 0000                                rseg    SUBR
21
22 0000 36      division      psha                ;hace espacio en el stack para guardar los
23 0001 37                                pshb                ;números a utilizar y los ASCII del
                                                ;resultado
24 0002 1830                                tsy
25 0004 3C                                pshx                ;hace backup de los registros
26 0005 183C                                pshy
27 0007 8645                                ldaa    #VARIAN
28 0009 B70001                            staa    FINSTACK    ;carga una posición anterior al comienzo del
                                                ;stack un número arbitrario
29
30
31
32 *****
33 *   SECCIÓN DE RECIBO DE NÚMEROS                         *
34 *   esta sección se encarga de tomar los números        *
35 *   ingresados usuario por stack                        *
36 *****
37
38 000C 18E605                                ldab    5,y                ;analiza el segundo número ingresado
39 000F C12F                                cmpb    #$2F                ;ve si es un operando o un dígito
40 0011 270D                                beq     loop                ;si es el operando, el segundo número es de
                                                ;un dígito
41 0013 860A                                ldaa    #$A                ;si es un dígito, el segundo número es de
                                                ;dos dígitos
42 0015 3D                                mul                        ;multiplica el dígito por 10, lo transforma
                                                ;a decenas
43 0016 18A604                            ldaa    4,y                ;toma las unidades del segundo número
44 0019 1B                                aba                        ;las suma a las decenas
45 001A 18A700                            staa    0,y                ;guarda el segundo número en hexadecimal en
                                                ;el stack
46 001D 7E0029                            jmp     loop1              ;analiza el primer número ingresado
47
48 0020 18A604      loop                  ldaa    4,y                ;toma el único dígito del segundo número
49 0023 18A700                            staa    0,y                ;guarda el segundo número en hexadecimal en
                                                ;el stack
50 0026 7E003D                            jmp     loop2              ;analiza el primer número ingresado
51
52 0029 18E608      loop1                 ldab    8,y                ;toma el valor 8 lugares después del stack
53 002C C145                            cmpb    #VARIAN            ; lo compara con un valor
54 002E 2721                            beq     loop3              ;si es ese valor, el primer número tiene un
                                                ;dígito
55 0030 860A                                ldaa    #$A                ;si no es ese valor, el primer número tiene
                                                ;dos dígitos
56 0032 3D                                mul                        ;multiplica el dígito por 10, lo transforma
                                                ;a decenas
57 0033 18A607                            ldaa    7,y                ;toma las unidades del primer número
58 0036 1B                                aba                        ;las suma a las decenas
59 0037 18A701                            staa    1,y                ;guarda el primer número en hexadecimal en
                                                ;el stack
60 003A 7E0063                            jmp     oper
61
62 003D 18E607      loop2                 ldab    7,y                ;toma el valor 8 lugares después del stack
63 0040 C145                            cmpb    #VARIAN            ; lo compara con un valor
64 0042 2716                            beq     loop4              ;si es ese valor, el primer número tiene un
                                                ;dígito

```

```

65 0044 860A          ldaa    #$A          ;si no es ese valor, el primer número tiene
66 0046 3D            mul          ;dos dígitos
67 0047 18A606        ldaa    6,y          ;multiplica el dígito por 10, lo transforma
68 004A 1B            aba          ;a decenas
69 004B 18A701        staa    1,y          ;toma las unidades del primer número
70 004E 7E0063        jmp     oper          ;las suma a las decenas
71                                     ;guarda el primer número en hexadecimal en
72 0051 18E607        ldab    7,y          ;el stack
73 0054 18E701        stab    1,y          ;toma el único dígito del primer número
74 0057 7E0063        jmp     oper          ;guarda el primer número en hexadecimal en
75 005A 18E606        ldab    6,y          ;el stack
76 005D 18E701        stab    1,y          ;toma el único dígito del primer número
77 0060 7E0063        jmp     oper          ;guarda el primer número en hexadecimal en
78                                     ;el stack
79
80
81 *****
82 *   SECCIÓN DE OPERACIÓN                               *
83 *   esta sección se encarga de dividir los números      *
84 *   recibiendo los por stack y devuelve el resultado en  *
85 *   el registro IX                                       *
86 *****
87
88 0063                oper          equ     *
89 0063 4F            clra
90 0064 18E600        ldab    0,y
91 0067 C100          cmpb    #0          ;si dividido por 0, error
92 0069 270C          beq     er
93 006B 8F            xgdx          ;tomo el valor del segundo número
94 006C 4F            clra
95 006D 18E601        ldab    1,y          ;toma el valor del primer número
96 0070 02            idiv          ;divide el primer número por el segundo
97 0071 CDEF00        stx     0,y          ;guarda el resultado en el stack
98 0074 7E0080        jmp     trans
99
100 0077 BD0000        er          jsr     error
101 007A 1838          puly          ;hace restore de los registros
102 007C 38            pulx
103 007D 33            pulb
104 007E 32            pula
105 007F 39            rts          ;vuelve al main program
106
107
108
109 *****
110 *   SECCIÓN DE TRANSFORMACIÓN A ASCII Y DISPLAY          *
111 *   esta sección se encarga de transformar el resultado *
112 *   en ASCII y mandarlo al display                        *
113 *****
114
115 0080 18EC00        trans        ldd     0,y          ;toma el resultado de la división
116 0083 C10A          cmpb    #$A          ;ve si el número se encuentra entre 0-9
117 0085 2B20          bmi     loop5        ;si se encuentra entre 0-9, pasa a ASCII ese
118                                     ;dígito
119 0087 18EC00        ldd     0,y          ;si es mayor a 9, toma nuevamente el
120                                     ;resultado
121 008A CE000A        ld      #$A
122 008D 02            idiv          ;divide el resultado por 10
123 008E CDEF00        stx     0,y          ;guarda el primer dígito del resultado
124 0091 18E700        stab    0,y          ;guarda el segundo dígito del resultado

```

```

123
124 0094 18A601      ldaa    1,y      ;toma el primer dígito del resultado
125 0097 C630        ldab    #$30
126 0099 1B          aba
127 009A 18A701      staa    1,y      ;lo convierte en ASCII
128 009D 18A600      ldaa    0,y      ;guarda el primer dígito en ASCII
129 00A0 1B          aba            ;toma el segundo dígito del resultado
130 00A1 18A700      staa    0,y      ;lo transforma en ASCII
131 00A4 7E00B0      jmp     disp      ;guarda el segundo dígito en ASCII
132
133 00A7 18A601      ldaa    1,y      loop5
134 00AA C630        ldab    #$30
135 00AC 1B          aba
136 00AD 18A700      staa    0,y
137
138 00B0 C600        ldab    #DI_END   ;toma el valor de la última posición del
139 00B2 CE0002      ldx     #2         ;display
140 00B5 18A600      ldaa    0,y      ;inicia contador de número a mostrar
141 00B8 BD0000      jsr     outchar   ;toma el dígito en ASCII
142 00BB 09          dex            ;lo manda al display
143 00BC 2705        beq     negativo  ;decremento el contador
144 00BE 5C          incb            ;si el contador es 0, analizo el signo
145 00BF 1808        iny            ;si no es cero el contador, toma el valor de
146 00C1 20F2        bra     repeat    ;la anterior posición del display
147
148 00C3 B60000      ldaa    flagn1    ;apunta al siguiente dígito
149 00C6 8101        cmpa    #1        ;repite proceso
150 00C8 2702        beq     negativo2 ;toma el valor del indicador de negatividad
151 00CA 2009        bra     negativo2 ;del primer número
152
153 00CC B60000      ldaa    flagn2    ;el primer número es negativo, analizo el
154 00CF 8101        cmpa    #1        ;segundo
155 00D1 270B        beq     negativo0  ;el primer número es positivo, analizo el
156 00D3 200C        bra     negativo01 ;segundo
157
158 00D5 B60000      ldaa    flagn2    ;toma el valor del indicador de negatividad
159 00D8 8101        cmpa    #1        ;del segundo número
160 00DA 2705        beq     negativo01 ;el segundo número es negativo, el resultado
161 00DC 2000        bra     negativo0  ;es positivo
162
163 00DE 7E00EB      jmp     theend    ;el segundo número es positivo, el resultado
164
165 00E1 C603        ldab    #DI_START ;es negativo
166 00E3 862D        ldaa    #$2D     ;termina la subrutina
167 00E5 BD0000      jsr     outchar   ;toma el valor de la primera posición del
168 00E8 7E00EB      jmp     theend    ;display
169
170 00EB 1838        puly            ;toma el valor ASCII del signo menos
171 00ED 38          pulx            ;lo manda al display
172 00EE 33          pulb            ;termina la subrutina
173 00EF 32          pula
174 00F0 39          rts              ;hace restore de los registros
175

```

```
176
177
178 00F1                                END
```

```
Errors:  None          #####
Bytes:   241           # divi #
CRC:     1816          #####
```

LINKOUT

```
#####
#
#      Micro Series Universal Linker V4.46C/DXT      28/Oct/120  12:21:31      #
#
#      Target CPU      = 68hc11                      #
#      List file       = linkout.lst                  #
#      Output file 1   = aout.a07                    #
#      Output format  = motorola                      #
#      Command line    = -f li11 (-ZCODE=2000 -ZVARS=3000 -ZCONST=4000      #
#                      -ZSUBR=4500 -ZSTACK=5500 -c68hc11 -Fmotorola -x      #
#                      -B) -A tp6j shownum error sumac restac prod divi    #
#                      -l linkout.lst                  #
#
#                      (c) Copyright IAR Systems 1993 #
#####
```

```
*****
*
*      CROSS REFERENCE      *
*
*****
```

There are no program entries.

```
*****
*
*      MODULE MAP          *
*
*****
```

FILE NAME : tp6j.r07
PROGRAM MODULE, NAME : tp6j

SEGMENTS IN THE MODULE
=====

CODE

Relative segment, address : 2000 - 21D5

SUBR

Relative segment, address : 4500 - 4530

ENTRIES	ADDRESS	REF BY MODULE
outchar	452A	shownum

error
sumac
restac
prod
divi

----- VARS

Relative segment, address : 3000 - 3009		
ENTRIES	ADDRESS	REF BY MODULE
counter	3000	shownum
msg	3008	shownum
flagn1	3005	sumac
		restac
		prod
		divi
flagn2	3006	sumac
		restac
		prod
		divi
flags	3001	Not referred to
flagr	3002	Not referred to

----- STACK

Relative segment, address : 5500 - 5563		
ENTRIES	ADDRESS	REF BY MODULE
stackP	5563	sumac
		restac
		prod
		divi

FILE NAME : shownum.r07
PROGRAM MODULE, NAME : shownum

SEGMENTS IN THE MODULE
=====

SUBR

Relative segment, address : 4531 - 4570		
ENTRIES	ADDRESS	REF BY MODULE
show	4531	tp6j

FILE NAME : error.r07
PROGRAM MODULE, NAME : error

SEGMENTS IN THE MODULE
=====

SUBR

Relative segment, address : 4571 - 4590		
ENTRIES	ADDRESS	REF BY MODULE
error	4571	shownum
		sumac
		restac
		prod
		divi

FILE NAME : sumac.r07
PROGRAM MODULE, NAME : sumac

SEGMENTS IN THE MODULE
=====

```
SUBR
Relative segment, address : 4591 - 46C6
    ENTRIES      ADDRESS      REF BY MODULE
    sumar        4591         tp6j

*****

FILE NAME : restac.r07
PROGRAM MODULE, NAME : restac

SEGMENTS IN THE MODULE
=====

SUBR
Relative segment, address : 46C7 - 47FD
    ENTRIES      ADDRESS      REF BY MODULE
    restar       46C7         tp6j

*****

FILE NAME : prod.r07
PROGRAM MODULE, NAME : prod

SEGMENTS IN THE MODULE
=====

SUBR
Relative segment, address : 47FE - 492C
    ENTRIES      ADDRESS      REF BY MODULE
    multi        47FE         tp6j

*****

FILE NAME : divi.r07
PROGRAM MODULE, NAME : divi

SEGMENTS IN THE MODULE
=====

SUBR
Relative segment, address : 492D - 4A1D
    ENTRIES      ADDRESS      REF BY MODULE
    division     492D         tp6j

*****
*
*      SEGMENTS IN DUMP ORDER      *
*
*****

SEGMENT      START ADDRESS      END ADDRESS      TYPE      ORG      P/N      ALIGN
=====
CODE          2000      -      21D5      rel      stc      pos      0
VARS          3000      -      3009      rel      stc      pos      0
CONST                Not in use      dse      stc      pos      0
SUBR          4500      -      4A1D      rel      stc      pos      0
STACK         5500      -      5563      rel      stc      pos      0

*****
*
*      END OF CROSS REFERENCE      *
*
*****
```

Errors: none
Warnings: none